HyperProve: Answer-Guided Hypergraph Expansion for Multi-Hop Question Answering

An Nguyen Phu[1]*, **Dung Nguyen Quang**[1]*, **Luu Hieu An**[1], **Linh Ngo Van**[1]†, **Trung Le**[2], **Thien Huu Nguyen**[3]

[1]Hanoi University of Science and Technology
[2]Monash University, [3]University of Oregon

Abstract

Multi-hop question answering often fails when retrieval treats evidence as isolated matches to the original question, since the facts needed to answer a complex question are usually connected through intermediate entities, relations, and constraints. We propose **HyperProve**, a retrieval-augmented QA framework that addresses this challenge by coupling question decomposition with answer-conditioned expansion over a hypergraph of atomic facts. HyperProve does not use atomic facts, hypergraphs, or iterative retrieval in isolation; instead, it carries intermediate answers and supporting hyperedges as retrieval state, then uses that state to bias the next local hypergraph expansion. This design enables HyperProve to construct coherent evidence chains for final answer generation while making the retrieval process stateful and fact-centered. Across multi-hop QA benchmarks, HyperProve achieves the best overall performance in our evaluation, outperforming the strongest baselines by an average relative improvement of 6.2% in answer accuracy and 4.9% in F1.

1 Introduction

Retrieval-augmented generation (Lewis et al., 2020) has become a standard way to ground language models in external evidence, but many retrieval pipelines still treat question answering as a one-shot relevance problem: given a question, retrieve passages that are independently similar to it, then ask a reader to generate the answer. This setup is better suited to questions whose answer-bearing evidence is directly aligned with the query. In multi-hop question answering, however, the crucial evidence is often distributed across facts connected through intermediate entities, relations, temporal constraints, or other bridge conditions (Trivedi et al., 2022; Yang et al., 2018; Ho et al., 2020). In such cases, the next useful fact may not be similar to the original question at all; it becomes relevant only after an earlier fact has established the right bridge or connection.

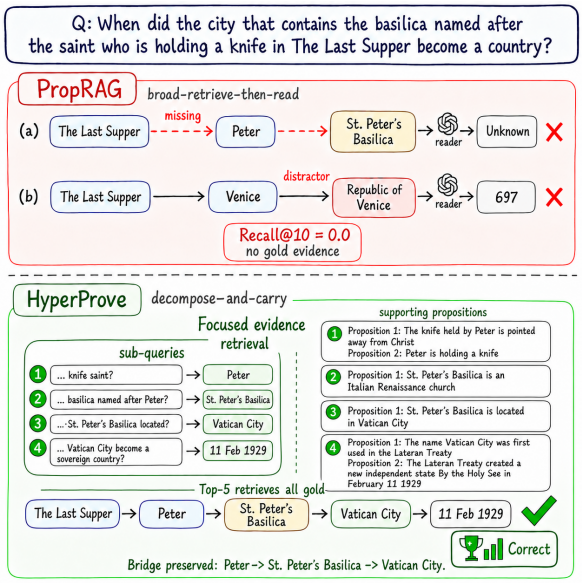


Figure 1: Case study of a 4-hop question: PropRAG fails to connect *Last Supper* to *Peter* and drifts to the distractor chain *Venice* → *Republic of Venice*; HyperProve succeeds because atomic-fact hyperedges explicitly preserve the bridge entities and correct path.

Prior work attacks this problem through graph structure and iterative retrieval, but its retrieval control differs from ours. HyperGraphRAG represents n-ary facts as hyperedges, yet matches nodes against the original question and applies a one-shot bipartite incidence expansion (Luo et al., 2025). HGRAG instead represents whole passages as hyperedges and combines entity- and passage-level relevance through fixed-query hypergraph diffusion (Wang et al., 2026). PropRAG preserves context in propositions and performs efficient, LLM-free beam search over proposition paths scored against the unchanged question (Wang and Han, 2025). These methods improve structured evidence

*Equal contribution
†Corresponding author: linhnv@soict.hust.edu.vn

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 4867–4882
October 24-29, 2026 ©2026 Association for Computational Linguistics

discovery, but graph adjacency or original-query relevance, rather than a validated intermediate answer and its support, determines how retrieval proceeds. As Figure 1 illustrates, a fixed-query proposition path can retain a globally similar distractor after the unresolved bridge has changed.

Iterative retrieval methods decompose questions, rewrite queries, or interleave retrieval with reasoning (Trivedi et al., 2023; Wang et al., 2025; Ye et al., 2025; Zhu et al., 2025). SG-FSM, for example, dynamically advances among sub-questions using textual state, while IRCoT and ChainRAG use intermediate reasoning or rewritten queries to guide later retrieval. Such systems adapt the query across hops, but they do not convert the resolved answer together with its supporting facts into a structural retrieval frontier. A later hop can therefore restart from surface similarity, lose the bridge evidence, or follow a plausible distractor entity.

We propose **HyperProve**, a retrieval-augmented QA framework that makes retrieval state explicit at the level of source-grounded atomic-fact hyperedges. Its key coupling is *resolved answer + supporting atomic facts → next-hop structural frontier*. HyperProve decomposes a question into dependent sub-queries, rewrites each unresolved relation with validated bridge answers, combines the carried frontier with fresh dense seeds, and performs bounded query-aware fact-to-fact expansion. Unlike one-shot incidence expansion, fixed-query diffusion, or textual answer propagation, this mechanism changes the next retrieval distribution using both the resolved answer and the evidence that supports it. In Figure 1, the resulting frontier preserves the hidden bridge evidence and keeps the chain on the correct path.

Our main contributions are summarized as follows:

- We propose **HyperProve**, an answer-conditioned hyperedge retrieval formulation that unifies graph-based and iterative RAG: each intermediate answer is carried with its supporting atomic-fact hyperedges as the structural frontier for subsequent hops.
- We design a query-biased local hypergraph expansion algorithm that retrieves over atomic-fact hyperedges, combining structural connectivity among evidence units with semantic alignment to each rewritten sub-query.
- Under a controlled retriever, reader, corpus, and evidence budget, we show that HyperProve improves multi-hop QA across three benchmarks over dense, structured, and iterative retrieval baselines.

2 Related Work

Graph-based RAG. Graph-based RAG improves over dense retrieval by modeling relationships among passages, entities, or atomic facts instead of scoring passages independently. HippoRAG2 builds graph memory with Personalized PageRank (Gutiérrez et al., 2025); PropRAG searches proposition paths to retain fine-grained context (Wang and Han, 2025); HyperGraphRAG represents n-ary facts as hyperedges (Luo et al., 2025); and HGRAG uses an entity–passage hypergraph for multi-hop QA (Wang et al., 2026). The same structural view extends beyond English open-domain QA: MaGiX builds a multi-granular cross-lingual graph with cross-synonym edges for English–Vietnamese RAG (Hieu et al., 2025), and MemORAI organizes conversational memory in a provenance-aware multi-relational graph traversed by Dynamic Weighted PageRank (Van et al., 2026). Cross-lingual retrievers trained with symmetric objectives further improve the underlying matching quality (Nguyen et al., 2025). However, they remain suboptimal for multi-hop QA because their retrieval stays conditioned on the original question, whereas HyperProve carries each intermediate answer and its supporting hyperedges across sub-queries to guide query-biased local expansion over atomic facts.

Iterative Retrieval. Iterative methods plan, retrieve, and refine evidence across multiple steps. IRCoT interleaves retrieval with chain-of-thought reasoning (Trivedi et al., 2023); SG-FSM advances through dynamically selected sub-questions using a textual finite-state process (Wang et al., 2025); T2RAG reasons over triples (Gong et al., 2026); HopRAG performs retrieve–reason–prune steps (Liu et al., 2025); and ChainRAG progressively rewrites queries to mitigate lost-in-retrieval errors (Zhu et al., 2025). KiRAG and UniRAG further adapt retrieval through knowledge-driven iteration and decomposition (Fang et al., 2025; Kim et al., 2025). HyperProve shares their hop-adaptive view, but changes the type of state: the validated answer and its supporting atomic facts define the next structural frontier, while the rewritten sub-query biases transitions within that bounded neighborhood.

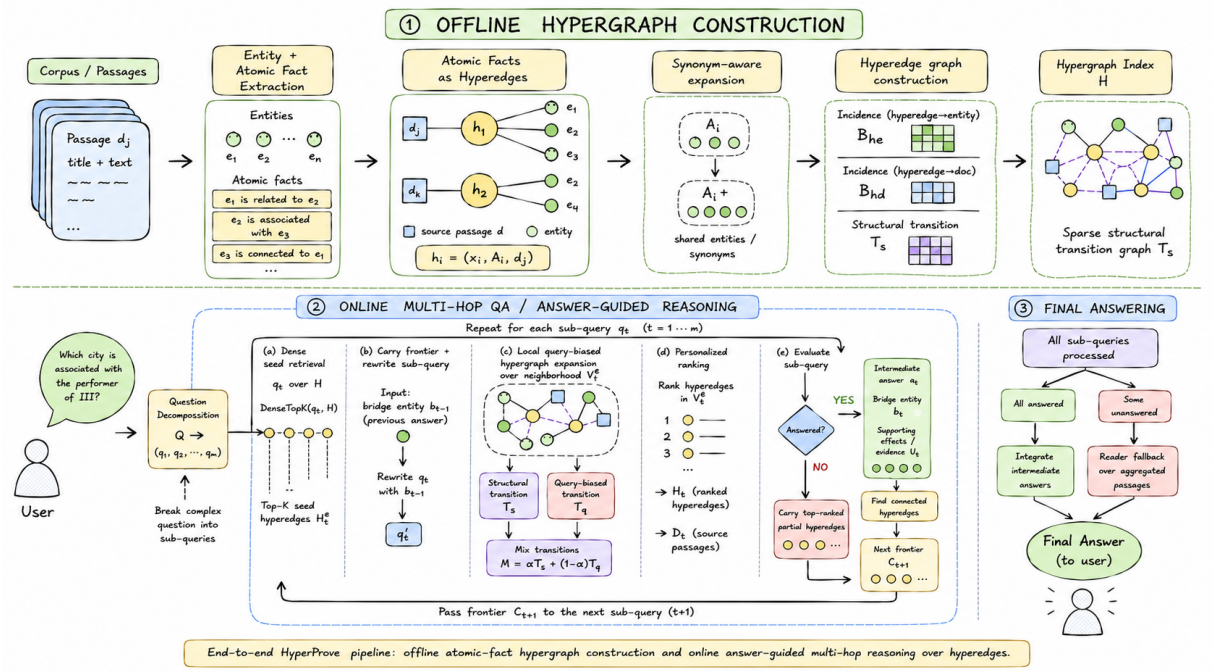


Figure 2: Overall HyperProve pipeline. The system builds an offline synonym-aware atomic-fact hypergraph from corpus passages, then performs online answer-guided multi-hop reasoning by decomposing a question into sub-queries, carrying intermediate answers as bridge entities, expanding over local hypergraph neighborhoods, and integrating the resulting evidence chain into a final answer.

This preserves evidence provenance across hops rather than carrying only text into a new retrieval call.

3 Methodology

Let $\mathcal{D} = \{d_j\}_{j=1}^{M_d}$ be document nodes and let $p_j = \text{text}(d_j)$ denote the passage stored at node d_j. Given question Q, our goal is to retrieve a compact set of connected evidence and generate answer a. We view a multi-hop question as a sequence of dependent sub-queries,

$$Q \to (q_1, q_2, \ldots, q_m),$$

where each q_t resolves at least one bridge entity, bridge relation, or final target attribute. The answer or evidence from q_{t-1} conditions the interpretation and retrieval frontier of q_t. HyperProve operationalizes this view by constructing a hyperedge-level knowledge structure and performing all intermediate reasoning directly over atomic-fact hyperedges. Figure 2 summarizes the resulting offline indexing and online answer-guided reasoning pipeline.

3.1 Atomic Facts as Hyperedges

As the first offline stage in Figure 2, HyperProve extracts entity nodes from each passage p_j,

$$\mathcal{E}_j \subseteq \mathcal{V} = \{e_a\}_{a=1}^{M_e},$$

where $m_a = \text{name}(e_a)$ is the textual mention associated with entity node e_a. The extracted nodes include named entities, dates, salient objects, and generic concepts that participate in meaningful relations. The passage is then decomposed into atomic facts. Let $\mathcal{H} = \{h_i\}_{i=1}^{N}$ be the resulting hyperedges and let $\sigma(i) = j$ map hyperedge h_i to its source document. We represent each atomic fact as

$$h_i = (x_i, \mathcal{A}_i, d_{\sigma(i)}),$$

where $x_i = \text{text}(h_i)$ is a self-contained factual statement and $\mathcal{A}_i \subseteq \mathcal{E}_{\sigma(i)}$ is the set of participating entity nodes. Structurally, its incidences are

$$h_i \sim \mathcal{A}_i \cup \{d_{\sigma(i)}\}.$$

This modeling choice addresses the limitations of binary triples. A triple represents a pairwise relation (s, r, o), but many facts involve more than two participants together with temporal, spatial, causal, or provenance qualifiers. Splitting such a fact into disconnected pairs can cause *context collapse*. PropRAG similarly motivates context-rich propositions over triples (Wang and Han, 2025); HyperProve additionally makes each atomic fact a hyperedge that connects all participating entity nodes and its source document. Extraction prompts are provided in Appendix A.1.

3.2 Knowledge Hypergraph Construction

The next offline stage turns the corpus into a synonym-aware hyperedge graph. Entity surface forms may differ across passages, so HyperProve embeds entity names m_a and creates a synonym map using cosine similarity. If two name embeddings exceed threshold η (0.8 in our experiments), their entity nodes are treated as neighboring mentions. The participant set is expanded as

$$\mathcal{A}_i^+ = \mathcal{A}_i \cup \mathrm{Syn}(\mathcal{A}_i).$$

The expanded set allows hyperedges to overlap even when passages use different surface forms for the same entity or concept.

We define hyperedge–entity incidence $B_{he} \in \{0,1\}^{N \times M_e}$ and hyperedge–document incidence $B_{hd} \in \{0,1\}^{N \times M_d}$:

$$B_{he}[i,a] = \mathbf{1}[e_a \in \mathcal{A}_i^+],$$
$$B_{hd}[i,j] = \mathbf{1}[\sigma(i) = j].$$

The entity and document degree matrices are

$$D_e[a,a] = \sum_i B_{he}[i,a],$$
$$D_d[j,j] = \sum_i B_{hd}[i,j].$$

The weighted structural adjacency between hyperedges is

$$W_s = B_{he} D_e^{-1} B_{he}^\top + B_{hd} D_d^{-1} B_{hd}^\top.$$

The first term connects hyperedges that share expanded entity nodes, while the second connects hyperedges from the same document. D_e^{-1} down-weights common entities and D_d^{-1} down-weights documents that yield many atomic facts. After removing self-loops, we row-normalize:

$$T_s = \mathrm{RowNormalize}(W_s - \mathrm{diag}(W_s)).$$

The resulting T_s is a sparse transition matrix over atomic-fact hyperedges; row normalization is applied only after self-transitions are removed. Documents remain provenance and reader context, while retrieval transitions operate over $\mathcal{H}$.

3.3 Query-biased Hyperedge Retrieval

For each query or sub-query, HyperProve uses dense retrieval only to select entry hyperedges. These seeds define where graph search begins; evidence is then ranked by navigating connected atomic facts in the hyperedge graph.

Starting from the current seeds, HyperProve builds local subgraph $G_t^\star$ by random-walk expansion over T_s, retaining at most 600 hyperedges connected through entities, synonym-expanded overlap, or shared provenance. Let $T_{s,t}^\star$ be the row-normalized restriction of T_s to $G_t^\star$. For candidate hyperedge h_k, define

$$c_{t,k} = \cos\big(\phi(q_t'), \phi(x_k)\big),$$
$$\mathcal{N}_{t,\theta}(i) = \{k \in \mathcal{N}_t(i) : c_{t,k} > \theta\},$$
$$Z_{t,i} = \sum_{\ell \in \mathcal{N}_{t,\theta}(i)} \exp(c_{t,\ell}/\tau).$$

The query-biased transition is

$$T_{q,t}^\star[i,k] = \begin{cases} \exp(c_{t,k}/\tau)/Z_{t,i}, & k \in \mathcal{N}_{t,\theta}(i), \\ 0, & \text{otherwise}, \end{cases}$$

where $\mathcal{N}_t(i)$ is the structural neighborhood of h_i in $G_t^\star$. Each row therefore assigns mass only to structurally connected hyperedges whose similarity to the current rewritten sub-query exceeds θ. If no candidate passes the gate, the row is zero and retrieval relies on the structural transition.

The final transition matrix mixes structural connectivity and query relevance:

$$M_t = \alpha T_{s,t}^\star + (1-\alpha) T_{q,t}^\star.$$

Here, α controls how much the walk follows the hypergraph structure versus the current query semantics. Personalized ranking is then performed over this local hyperedge graph:

$$\boldsymbol{\pi}_t^{(r+1)} = (1-\gamma) u_t^\star + \gamma M_t^\top \boldsymbol{\pi}_t^{(r)},$$

where r indexes power iteration and $u_t^\star$ resets to the current seeds. We use $\tau = 1$, $\theta = 0.4$, $\alpha = 0.5$, and $\gamma = 0.85$ throughout. The resulting $\boldsymbol{\pi}_t$ ranks atomic-fact hyperedges; documents are recovered only after hyperedge ranking through the source map σ.

3.4 Answer-Guided Hyperedge Reasoning

HyperProve decomposes a question into dependent sub-queries and carries evidence across them as a hyperedge frontier; Algorithm 1 gives the procedure. Given $Q \to (q_1, \ldots, q_m)$, let $\mathcal{C}_1 = \emptyset$ and, for $t > 1$, let $\mathcal{C}_t \subseteq \mathcal{H}$ be the frontier produced after hop $t-1$. Each hop uses at most $K_c = 10$ carried-frontier seeds and $K_d = 10$ fresh dense

Algorithm 1 HyperProve main algorithm.

Require: Hyperedge index $(\mathcal{H}, \mathcal{V}, \mathcal{D}, T_s)$, question Q, $K_c = K_d = 10$
Ensure: Final answer a
$(q_1, \ldots, q_m) \leftarrow \textsc{Decompose}(Q)$
$\mathcal{M}_H, \mathcal{M}_D, \mathcal{C}_1, \mathcal{R} \leftarrow \emptyset$
$\mathcal{M}_H$: hyperedge memory; $\mathcal{M}_D$: document memory
for $t = 1$ to m **do**
  if $t = 1$ **then**
    $q'_t \leftarrow q_t$; $\mathcal{S}_t \leftarrow \textsc{DenseTopK}(q'_t, \mathcal{H}, K_d)$
  else
    $q'_t \leftarrow \textsc{Rewrite}(q_t, \mathcal{R})$
    $\mathcal{S}_t \leftarrow \textsc{TopK}(\mathcal{C}_t, K_c) \cup \textsc{DenseTopK}(q'_t, \mathcal{H}, K_d)$
  end if
  retrieve $\mathcal{H}_t$, scores $\boldsymbol{\pi}_t$, passages $\mathcal{P}_t$ from $\mathcal{S}_t$
  $\mathcal{M}_H \leftarrow \mathcal{M}_H \cup \{(\mathcal{H}_t, \boldsymbol{\pi}_t)\}$
  $\mathcal{M}_D \leftarrow \textsc{MaxMerge}(\mathcal{M}_D, \mathcal{P}_t)$
  $(s_t, a_t, \mathcal{U}_t) \leftarrow \textsc{Evaluate}(q_t, \mathcal{H}_t, \mathcal{P}_t)$
  if $s_t = \textsc{Answered}$ **then**
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(q_t, a_t)\}$
    $b_t \leftarrow \textsc{BridgeEntity}(a_t)$; $\mathcal{E}_t \leftarrow \textsc{Entities}(\mathcal{U}_t)$
    $\mathcal{C}_{t+1} \leftarrow \textsc{FindHyperedges}(\{b_t\} \cup \mathcal{E}_t, \mathcal{H})$
  else
    $\mathcal{C}_{t+1} \leftarrow \textsc{TopK}(\mathcal{H}_t, \boldsymbol{\pi}_t, K_c)$
  end if
end for
if all sub-queries are answered **then**
  $a \leftarrow \textsc{IntegrateAnswers}(Q, \mathcal{R})$
else
  $\mathcal{P}_{10} \leftarrow \textsc{TopKPassages}(\mathcal{M}_H, \mathcal{M}_D, 10)$
  $a \leftarrow \textsc{ReaderFallback}(Q, \mathcal{P}_{10}, \mathcal{M}_H)$
end if
return a

seeds. Fresh seeds allow retrieval to escape a disconnected or incorrect local neighborhood. Let $\mathcal{F}_t$ denote the fresh seeds; then

$$\mathcal{F}_t = \mathrm{DenseTopK}(q'_t, \mathcal{H}, K_d),$$
$$\mathcal{S}_t = \begin{cases} \mathcal{F}_t, & t = 1, \\ \mathrm{TopK}(\mathcal{C}_t, K_c) \cup \mathcal{F}_t, & t > 1. \end{cases}$$

For the first hop, $q'_1 = q_1$. For $t > 1$, let $\mathcal{R}_{t-1} = \{(q_s, a_s) : s < t,\ s_s = \textsc{Answered}\}$ be the resolved question–answer memory. Then $q'_t = \mathrm{Rewrite}(q_t, \mathcal{R}_{t-1})$, which replaces referring expressions with validated intermediate answers while preserving the unresolved relation. If $\mathcal{R}_{t-1}$ is empty, $q'_t = q_t$.

When a sub-query is answered, bridge entity b_t and the entities in supporting hyperedges $\mathcal{U}_t$ define the next frontier. Let $\mathcal{E}_t = \mathrm{Entities}(\mathcal{U}_t)$; then

$$\mathcal{C}_{t+1} = \mathrm{FindHyperedges}\big(\{b_t\} \cup \mathcal{E}_t, \mathcal{H}\big).$$

If the sub-query is unresolved, HyperProve carries the top-K_c hyperedges under $\boldsymbol{\pi}_t$ as partial evidence.

Reader fallback. If every sub-query is answered, $\mathrm{IntegrateAnswers}(Q, \mathcal{R})$ produces the final answer. Otherwise, ReaderFallback max-merges evidence across hops and invokes the same reader LLM once with the original question. Given scored hyperedge memory $\mathcal{M}_H = \{(\mathcal{H}_t, \boldsymbol{\pi}_t)\}_{t=1}^{m}$, document d_j receives score

$$s(d_j) = \max_{\substack{t \in [m],\ h_i \in \mathcal{H}_t \\ \sigma(i) = j}} \pi_t(h_i).$$

Let $\mathcal{P}_{10}$ contain the ten documents with highest $s(d_j)$. The fallback reader receives Q, their passage texts p_j, and the corresponding ranked atomic facts from $\mathcal{M}_H$. Appendix A.1.6 and A.2 provide further details.

4 Experiments

4.1 Experimental Settings

Datasets and Metrics. We evaluate HyperProve on three open-domain multi-hop question answering benchmarks: MuSiQue (Trivedi et al., 2022), HotpotQA (Yang et al., 2018), and 2WikiMultiHopQA (Ho et al., 2020). We report three answer-quality metrics: LLM-as-a-judge answer accuracy, Exact Match (EM), and token-level F1.

Baselines. We compare HyperProve against three families of retrieval-augmented baselines. First, the dense retrieval family includes Contriever (Izacard et al., 2022) and BGE-M3 (Chen et al., 2024). Second, structured retrieval includes graph- and structure-based RAG methods: HippoRAG2 (Gutiérrez et al., 2025), PropRAG (Wang and Han, 2025), HyperGraphRAG (Luo et al., 2025), and HGRAG (Wang et al., 2026). Third, the iterative retrieval family includes IRCoT (Trivedi et al., 2023), T2RAG (Gong et al., 2026), HopRAG (Liu et al., 2025), and ChainRAG (Zhu et al., 2025).

Implementation Details. For all methods in the main experiments and analysis, we use BGE-M3 as the retriever and GPT-OSS-20B as the backbone LLM. Apart from this shared model setup, each baseline follows its original implementation, including its hyperparameter configurations. Table 1 reports the mean and standard deviation over three runs under this controlled setup.

4.2 Main Results

Dense retrieval lacks evidence state for multi-hop QA. Although BGE-M3 improves over Contriever, it remains far behind HyperProve: on MuSiQue, 51.50 accuracy and 39.70 F1 versus 71.60 and 59.23; on 2Wiki, 72.18 and 54.50 versus 91.40 and 80.01. Even on HotpotQA, where dense retrieval is stronger, BGE-M3 is still 23.18

Table 1: Mean ± standard deviation over three runs on multi-hop QA benchmarks. We report LLM-as-a-judge answer accuracy, Exact Match (EM), and token-level F1. Best means are in **bold**; second-best means are underlined.

	MuSiQue			HotpotQA			2Wiki		
Method	Acc.	EM	F1	Acc.	EM	F1	Acc.	EM	F1
Dense Retrieval									
Contriever	47.80 ± 0.28	22.30 ± 0.30	32.10 ± 0.26	55.55 ± 0.21	44.40 ± 0.24	57.50 ± 0.21	69.60 ± 0.20	38.50 ± 0.24	43.70 ± 0.21
BGE-M3	51.50 ± 0.25	27.80 ± 0.27	39.70 ± 0.24	68.12 ± 0.19	47.80 ± 0.22	61.80 ± 0.19	72.18 ± 0.18	48.20 ± 0.22	54.50 ± 0.19
Structured Retrieval									
HippoRAG2	52.00 ± 0.36	37.20 ± 0.40	48.60 ± 0.34	81.70 ± 0.29	62.70 ± 0.33	75.50 ± 0.29	86.40 ± 0.28	65.00 ± 0.34	71.00 ± 0.29
PropRAG	50.90 ± 0.30	29.30 ± 0.33	46.73 ± 0.29	85.80 ± 0.24	57.80 ± 0.28	73.79 ± 0.24	77.50 ± 0.23	58.00 ± 0.29	68.78 ± 0.24
HyperGraphRAG	55.13 ± 0.43	42.20 ± 0.47	53.80 ± 0.40	77.89 ± 0.34	63.90 ± 0.39	73.80 ± 0.33	77.72 ± 0.33	68.30 ± 0.40	71.30 ± 0.33
HGRAG	58.60 ± 0.39	41.19 ± 0.43	55.04 ± 0.37	80.10 ± 0.31	54.90 ± 0.36	69.43 ± 0.30	85.00 ± 0.30	64.10 ± 0.37	74.10 ± 0.31
Iterative Retrieval									
IRCoT	56.67 ± 0.64	34.10 ± 0.72	47.60 ± 0.59	85.23 ± 0.49	55.70 ± 0.56	71.20 ± 0.47	83.65 ± 0.47	60.70 ± 0.57	74.30 ± 0.47
T2RAG	54.67 ± 0.47	34.30 ± 0.52	45.60 ± 0.44	81.33 ± 0.36	54.20 ± 0.42	67.30 ± 0.35	83.75 ± 0.35	66.70 ± 0.43	74.40 ± 0.36
HopRAG	67.18 ± 0.72	42.40 ± 0.80	54.90 ± 0.66	85.51 ± 0.55	61.10 ± 0.63	68.26 ± 0.52	86.66 ± 0.53	62.00 ± 0.64	76.06 ± 0.52
ChainRAG	60.01 ± 0.66	41.20 ± 0.74	54.15 ± 0.61	85.46 ± 0.50	60.70 ± 0.58	74.66 ± 0.48	82.48 ± 0.49	63.30 ± 0.60	74.39 ± 0.49
HyperProve	**71.60 ± 0.58**	**42.83 ± 0.64**	**59.23 ± 0.54**	**91.30 ± 0.43**	**63.92 ± 0.49**	**76.64 ± 0.41**	**91.40 ± 0.42**	**69.70 ± 0.51**	**80.01 ± 0.43**

accuracy points lower. These gaps show that independent dense matching can retrieve locally relevant passages, but lacks persistent evidence state for later hops once a bridge entity or constraint has been resolved.

Static structure and iterative retrieval are complementary but incomplete. The strongest baseline alternates between Structured Retrieval and Iterative Retrieval methods: HopRAG leads accuracy on MuSiQue with 67.18 and 2Wiki with 86.66, while PropRAG leads HotpotQA accuracy with 85.80; HGRAG gives the strongest MuSiQue F1 with 55.04, and HyperGraphRAG gives the strongest baseline EM on HotpotQA with 63.90 and 2Wiki with 68.30. This uneven pattern reflects the limits of both families. Static structure helps connect evidence but uses a mostly fixed frontier that is not conditioned on intermediate answers, while iterative retrieval adapts the query across hops but can drift from the evidence that justified the previous hop.

HyperProve achieves the best evaluated performance. HyperProve has the highest mean on every reported metric across all three benchmarks. It improves answer accuracy over HopRAG by 4.42 points on MuSiQue and 4.74 points on 2Wiki; on HotpotQA, it improves over PropRAG by 5.50 accuracy points and over HippoRAG2 by 1.14 F1 points. The broad accuracy and F1 gains are larger than the observed run-level variation. We nevertheless avoid over-interpreting two narrow EM margins: 42.83 versus 42.40 on MuSiQue and 63.92 versus 63.90 on HotpotQA are within run-level variation and do not establish statistically resolved improvements. Overall, the results support using intermediate answers to define a structural frontier over atomic-fact hyperedges in addition to rewriting later queries.

Variant	**Overall**	**2-hop**	**3-hop**	**4-hop**
HyperProve	**71.60**	**78.00**	**72.96**	**49.40**
w/o bridge entity	64.50	69.11	67.41	44.58
w/o fresh dense seed	61.30	71.24	59.18	34.34
w/o decomposition	51.28	58.11	49.05	33.73
w/o T_s	62.13	67.02	64.65	42.06
w/o query transition	58.17	65.20	58.60	35.40

Table 2: Accuracy breakdown of HyperProve ablations on MuSiQue by reasoning depth. Accuracy is computed using judge verdicts over 1000 questions: 518 2-hop, 316 3-hop, and 166 4-hop examples.

5 Analysis

We analyze HyperProve from four perspectives: component ablations, evidence-chain faithfulness, efficiency, and qualitative behavior. Together, these analyses examine which mechanisms drive multi-hop performance, whether it accumulates connected evidence across hops, whether HyperProve improves accuracy with reasonable cost, and how it handles hidden bridge entities and multi-entity reasoning.

5.1 Retrieval Component Breakdown

The answer-guided ablations in Table 2 show that HyperProve relies on both bridge propagation and fresh query-specific entry points. Removing bridge entities lowers overall accuracy from 71.60 to 64.50, with a substantial drop on 4-hop questions from 49.40 to 44.58, suggesting that intermediate answers provide cross-hop links that dense similarity alone may miss. Removing fresh dense seeds

Method	Recall@10			Avg Gain
	MuSiQue	HotpotQA	2Wiki	After Hop 1
HyperProve	82.31	96.20	98.15	+41.24
ChainRAG	67.42	89.45	75.85	+26.35
HopRAG	66.63	82.82	84.17	+25.98

Table 3: Evidence chain faithfulness across multi-hop benchmarks. Dataset columns report Recall@10, computed as the average fraction of gold passages retrieved in the top 10 results. Avg Gain after Hop 1 measures the average increase in cumulative gold-passage coverage after the first hop.

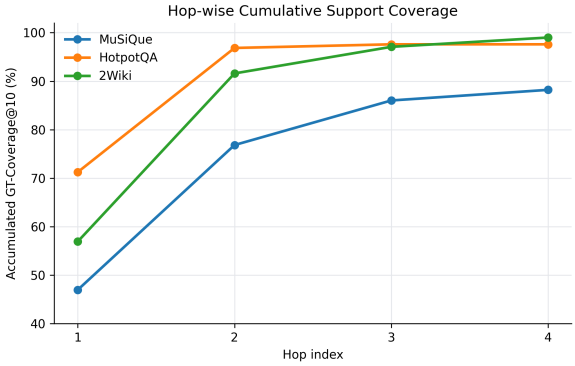


Figure 3: Hop-wise cumulative support coverage. Cumulative gold-passage coverage increases across hops, showing that later sub-queries progressively recover missing support evidence.

is also especially harmful for deeper reasoning: although 2-hop accuracy remains comparable, 4-hop accuracy falls to 34.34, indicating that the carried frontier alone can trap retrieval inside the current hyperedge neighborhood. The largest degradation comes from removing decomposition, which reduces overall accuracy to 51.28 and confirms that complex questions are better solved through dependent sub-queries than through a single retrieval step.

The transition-component ablations show that both T_s and $T^{\star}_{q,t}$ are necessary for effective hyperedge reasoning. Removing the structural transition T_s produces a pattern similar to removing bridge entities, but with a larger overall drop to 62.13, indicating that structural connectivity is crucial for maintaining valid evidence paths across hops. Removing the query-conditioned transition $T^{\star}_{q,t}$ causes an even sharper decline to 58.17 overall and 35.40 on 4-hop questions. This shows that structure alone is not enough: without query-aware transition, HyperProve no longer performs targeted graph reasoning and may expand along connected but irrelevant hyperedges.

5.2 Evidence Chain Faithfulness

Table 3 evaluates evidence quality under the same top-10 retrieval budget across MuSiQue, HotpotQA, and 2Wiki, rather than only reporting answer accuracy. HyperProve achieves the highest Recall@10 on all three benchmarks, with 82.31 on MuSiQue, 96.20 on HotpotQA, and 98.15 on 2Wiki. These results improve over the strongest iterative baselines by 14.89, 6.75, and 13.98 points, respectively. HyperProve also obtains the largest average gain after the first hop, +41.24 versus +26.35 for ChainRAG and +25.98 for HopRAG. Thus, additional hops are not simply adding more context: they recover missing gold passages while preserving a stronger final support set than both iterative baselines.

Figure 3 further isolates the hop-wise behavior of HyperProve. Across all three benchmarks, each decomposed hop increases or maintains cumulative support coverage, with especially large gains after the first hop. Because the curve measures accumulated gold-passage coverage at Recall@10, these increases reflect newly recovered supporting evidence rather than longer reasoning traces or repeated retrievals.

5.3 Efficiency Analysis

Offline construction. HyperProve's measured index uses 23,312 calls and 20.8M tokens, close to PropRAG's reported 23,312 calls and 21.1M tokens and HyperGraphRAG's normalized 20.9M-token estimate; entity-only HGRAG is cheaper at an estimated 5.7M tokens. Passage-local extraction can be parallelized and amortized over queries, although these workloads reflect different extraction objectives. Overall, HyperProve has comparable offline cost to PropRAG and HyperGraphRAG, while the online results below show that this one-time investment yields a favorable accuracy–efficiency trade-off.

Online inference. HyperProve uses 9.6M online tokens and 31.9K seconds for 1,000 MuSiQue questions. It improves accuracy over HopRAG by 4.42 points while using 26.5M fewer tokens and 13.3K fewer seconds. Against ChainRAG, it gains 11.59 points while using 5.8M fewer tokens and 5.1K fewer seconds. The lighter BGE-M3, IRCoT, and T2RAG systems remain substantially cheaper, but trail HyperProve by 20.10, 14.93, and 16.93 accuracy points, respectively. Thus, HyperProve oc-

System	Top-10 Coverage	Retrieval / Reasoning Behavior	Final Answer
HyperProve	4 / 4	Separately resolves *Pathology* → *San Diego* and *Still Swingin'* → *Papa Roach* → *California*, then applies the rank query to *San Diego* under the *California* urban-area constraint.	third-largest
ChainRAG	2 / 4	Retrieves evidence for *Pathology* → *San Diego* and *Still Swingin'* → *Papa Roach*, but misses the facts needed to bind *Papa Roach* → *Vacaville, California* and *San Diego urban area* → *third-largest in California*. Its reader is then pulled to a nearby city-rank answer.	2nd

Table 4: Qualitative comparison on a multi-entity MuSiQue example. HyperProve retrieves the complete evidence chain and preserves the urban-area constraint, while ChainRAG retrieves only partial evidence and confuses city population rank with urban-area rank.

(a) Offline construction

Method	Extraction	Calls	Tokens	Cost source
HyperGraphRAG	Facts+entities	11,656	20.9M	Normalized est.
PropRAG	Entities+props.	23,312	21.1M	Reported measure
HGRAG	Entities	11,656	5.7M	Normalized est.
HyperProve	Entities+facts	23,312	20.8M	Measured artifacts

(b) Online inference

Method	Acc.	Tokens	Lat. (s)
BGE-M3	51.50	1.7M	4.1K
IRCoT	56.67	4.4M	3.8K
T2RAG	54.67	2.4M	9.6K
HopRAG	67.18	36.1M	45.2K
ChainRAG	60.01	15.4M	37.0K
HyperProve	71.60	9.6M	31.9K

Table 5: Offline and online efficiency on MuSiQue. Panel (a) explicitly separates measured or reported costs from estimates normalized to the 11,656-passage corpus and the same token accounting. Panel (b) reports measured inference over 1,000 questions. All token and latency values exclude answer-judge cost.

cupies a middle accuracy–token–latency regime: more computation than lightweight retrieval, but bounded below the two strongest high-cost iterative baselines evaluated here.

5.4 Qualitative Analysis

Table 4 presents a multi-entity MuSiQue case with two initially separate evidence chains. One chain resolves *Pathology* to *San Diego*; the other resolves *Still Swingin'* to *Papa Roach* and then connects the band to *Vacaville, California*. Answering the question requires more than recovering these entities: the system must join the two chains and apply the requested rank to the *San Diego urban area within California*, rather than to San Diego as a city.

HyperProve retrieves all four gold passages. At each hop, query rewriting introduces the resolved bridge entity, while the carried hyperedge frontier keeps that entity attached to its supporting fact and source. The first branch therefore preserves the semantic target *San Diego urban area*; the second supplies the geographic constraint *California*. When the branches meet, the final retrieval is conditioned on both pieces of state, allowing the reader to select *third-largest*. In this example, query rewriting provides cross-hop reach, whereas structural propagation preserves provenance and prevents the rank constraint from drifting to a related but different entity type.

ChainRAG covers only two of the four gold passages. It recovers *Pathology* → *San Diego* and *Still Swingin'* → *Papa Roach*, but misses both the California-binding fact and the evidence that ranks the San Diego urban area. Its answer *2nd* is thus not merely caused by fewer retrieved passages: the incomplete chain leaves the reader with a nearby city-population statistic whose entity scope does not match the question. The comparison illustrates why multi-hop retrieval must preserve relation type, modifiers, and provenance in addition to entity names. Although this single case is not evidence of aggregate superiority by itself, it provides a mechanism-level explanation for the evidence-coverage gains in Section 5.2.

6 Conclusion

We presented HyperProve, an answer-guided hypergraph expansion framework. HyperProve decomposes complex questions into dependent subqueries, carries intermediate answers and supporting hyperedges as retrieval state, and performs query-biased local expansion over atomic-fact hyperedges that preserve multi-entity context and source provenance. Across three multi-hop QA benchmarks, HyperProve achieves the strongest performance among dense, structured, and iterative retrieval baselines. These results show the value of combining stateful query control with structured

evidence accumulation.

Limitations

HyperProve has three limitations. First, it incurs substantial LLM cost during offline extraction and online reasoning; future work could distill these components using recent trajectory, representation, or cross-tokenizer alignment methods (Chi et al., 2026; Dao et al., 2026a; Vu et al., 2026; Nguyen et al., 2026). Second, LLM-as-a-judge accuracy may miss subtle reasoning, evidence-sufficiency, and faithfulness errors, motivating human evaluation and stronger evidence-chain metrics. Finally, our experiments are limited to fixed open-domain QA benchmarks with stable corpora and predefined answers, which does not fully capture noisy, incomplete, conflicting, multilingual, or temporally changing evidence.

Acknowledgments

This research was funded by Vingroup Innovation Foundation (VINIF) under project code VINIF.2026.DA117.

References

Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2318–2335, Bangkok, Thailand. Association for Computational Linguistics.

Pham Khanh Chi, Quoc Phong Dao, Thuat Nguyen, Linh Ngo Van, Trung Le, and Thanh Hong Nguyen. 2026. MTA: Multi-granular trajectory alignment for large language model distillation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32669–32684. Association for Computational Linguistics.

Quoc Phong Dao, Hoang Son Nguyen, Pham Khanh Chi, Tung Nguyen, Linh Ngo Van, Nguyen Thi Ngoc Diep, and Trung Le. 2026a. SRA: Span representation alignment for large language model distillation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32961–32975. Association for Computational Linguistics.

Quoc Phong Dao, Hoang Son Nguyen, Pham Khanh Chi, Linh Ngo Van, Nguyen Thi Ngoc Diep, Thien Huu Nguyen, and Trung Le. 2026b. TALAS: Teacher-anchored layer alignment with adaptive sharpness-aware minimization for embedding distillation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32705–32723, San Diego, California, United States. Association for Computational Linguistics.

Jinyuan Fang, Zaiqiao Meng, and Craig Macdonald. 2025. KiRAG: Knowledge-driven iterative retriever for enhancing retrieval-augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18969–18985. Association for Computational Linguistics.

Shengbo Gong, Xianfeng Tang, Qi He, Carl Yang, and Wei Jin. 2026. Beyond chunks and graphs: Retrieval-augmented generation through triplet-driven thinking. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 26282–26308, San Diego, California, United States. Association for Computational Linguistics.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 21497–21515. PMLR.

Nguyen Manh Hieu, Vu Lam Anh, Hung Pham Van, Nam Le Hai, Linh Ngo Van, Nguyen Thi Ngoc Diep, and Thien Huu Nguyen. 2025. MaGiX: A multi-granular adaptive graph intelligence framework for enhancing cross-lingual RAG. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5202–5219. Association for Computational Linguistics.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625. International Committee on Computational Linguistics.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*.

Gun Il Kim, Jong Wook Kim, and Beakcheol Jang. 2025. UniRAG: A unified RAG framework for knowledge-intensive queries with decomposition, break-down reasoning, and iterative rewriting. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 18795–18810, Suzhou, China. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020.

Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. 2025. HopRAG: Multi-hop reasoning for logic-aware retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 1897–1913. Association for Computational Linguistics.

Haoran Luo, Haihong E, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, and Anh Tuan Luu. 2025. HyperGraphRAG: Retrieval-augmented generation via hypergraph-structured knowledge representation. In *Advances in Neural Information Processing Systems*, volume 38, pages 168965–168993. Curran Associates, Inc.

Toan Ngoc Nguyen, Nam Le Hai, Nguyen Doan Hieu, Dai An Nguyen, Linh Ngo Van, Thien Huu Nguyen, and Sang Dinh. 2025. Improving vietnamese-english cross-lingual retrieval for legal and general domains. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pages 142–153. Association for Computational Linguistics.

Truong Nguyen, Phi Van Dat, Ngan Nguyen, Linh Ngo Van, Trung Le, and Thanh Hong Nguyen. 2026. CTPD: Cross tokenizer preference distillation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(44):37783–37790.

Thang Duc Tran, Minh-Phuc Truong, Thuan Do Phan, and Linh Ngo Van. 2026. SAMD: Span-aware matryoshka distillation for cross-tokenizer embedding models. *Machine Learning*, 115(6):143.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10014–10037. Association for Computational Linguistics.

Minh-Phuc Truong, Hai An Vu, Tu Vu, Nguyen Thi Ngoc Diep, Linh Ngo Van, Thien Huu Nguyen, and Trung Le. 2025. EMO: Embedding model distillation via intra-model relation and optimal transport alignments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 7594–7606, Suzhou, China. Association for Computational Linguistics.

Hung Pham Van, Nguyen Manh Hieu, Khang Pham Tran Tuan, Nam Le Hai, Linh Ngo Van, Nguyen Thi Ngoc Diep, and Trung Le. 2026. MemORAI: Memory organization and retrieval via adaptive graph intelligence for LLM conversational agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 28235–28253. Association for Computational Linguistics.

Duc Trung Vu, Pham Khanh Chi, Dat Phi Van, Linh Ngo Van, Dinh Viet Sang, and Trung Le. 2026. DWA-KD: Dual-space weighting and time-warped alignment for cross-tokenizer knowledge distillation. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3513–3527. Association for Computational Linguistics.

Changjian Wang, Weihong Deng, Weili Guan, Quan Lu, and Ning Jiang. 2026. Cross-granularity hypergraph retrieval-augmented generation for multi-hop question answering. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(39):33368–33376.

Jingjin Wang and Jiawei Han. 2025. PropRAG: Guiding retrieval with beam search over proposition paths. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 6212–6227, Suzhou, China. Association for Computational Linguistics.

Xiaochen Wang, Junqing He, Liang Chen, Gholamreza Haffari, Yiru Wang, Zhe Yang, Xiangdi Meng, Kunhao Pan, and Zhifang Sui. 2025. SG-FSM: A self-guiding zero-shot prompting paradigm for multi-hop question answering based on finite state machine. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6040–6052. Association for Computational Linguistics.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics.

Linhao Ye, Lang Yu, Zhikai Lei, Qin Chen, Jie Zhou, and Liang He. 2025. Optimizing question semantic space for dynamic retrieval-augmented multi-hop question answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17814–17824. Association for Computational Linguistics.

Rongzhi Zhu, Xiangyu Liu, Zequn Sun, Yiwei Wang, and Wei Hu. 2025. Mitigating lost-in-retrieval problems in retrieval augmented multi-hop question answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22362–22375. Association for Computational Linguistics.

A Appendix

This appendix provides additional details for HyperProve, including the prompts used for offline hypergraph construction and online answer-guided reasoning, scalability limitations, dataset statistics, positioning relative to prior work, and two case studies.

A.1 Prompt Templates

This section lists the prompt templates used by HyperProve for offline hypergraph construction, online answer-guided retrieval, answer generation, and automatic evaluation. All prompts are designed to produce structured outputs, which simplifies downstream parsing and makes the reasoning pipeline more reproducible.

A.1.1 Entity Extraction

```
Task:
Given a passage, identify the entities that are
needed to represent the factual content of the
passage.

Output format:
Return only a valid JSON object with the
following structure:
{
  "entities": ["entity_1", "entity_2", "..."]
}

Guidelines:
1. Include all named entities mentioned in the
passage.
2. Include all dates, temporal expressions, and
events.
3. Include generic concepts when they are
central to the passage topic.
4. Include entities that participate in
meaningful relations with the extracted entities.

5. Preserve the surface form used in the passage
 whenever possible.
6. Do not return an empty list.
7. Do not include explanations outside the JSON
object.

Example passage:
Radio City is India's first private FM radio
station and was started on
3 July 2001. It plays Hindi, English and
regional songs. Radio City recently
forayed into New Media in May 2008 with the
launch of a music portal,
PlanetRadiocity.com, that offers music-related
news, videos, songs, and other
features.

Example output:
{
  "entities": [
    "Radio City",
    "India",
    "private FM radio station",
    "3 July 2001",
    "Hindi",
    "English",
    "regional songs",
    "New Media",
    "May 2008",
    "PlanetRadiocity.com",
    "music portal",
    "music-related news",
    "videos",
    "songs"
  ]
}

Input:
Passage: {passage}
```

A.1.2 Atomic-Fact Hyperedge Extraction

```
Task:
Given a passage and a predefined list of
entities, decompose the passage into atomic
facts. Each atomic fact should express one self-
contained claim and explicitly specify the
entities involved in that fact.

Output format:
Return only a valid JSON object with the
following structure:
{
  "hyperedges": [
    {
      "text": "a complete atomic fact",
      "entities": ["entity_1", "entity_2",
"..."]
    }
  ]
}

Guidelines:
1. Each hyperedge must contain exactly one
factual claim or relation.
2. Each hyperedge must be understandable without
 requiring the original passage.
3. Use only entities from the provided entity
list. Do not introduce new entities.
4. Make the relation among entities explicit and
 unambiguous.
5. Preserve temporal, causal, comparative, and
modifier information when it affects the meaning
 of the fact.
6. When a statement involves more than two
entities, keep them together in a single
hyperedge rather than splitting the fact into
disconnected binary relations.
7. Include all relevant entities from the
provided entity list in both the hyperedge text
and the entity array.
8. Ensure that the set of hyperedges covers all
important factual information in the passage.
9. Do not include explanations outside the JSON
object.

Example passage:
In 2020, after Apple launched the M1 chip, major
 software companies like Adobe optimized their
applications, improving performance by up to 80
percent compared to Intel-based Macs.

Entity list:
[
```

```
  "Apple",
  "M1 chip",
  "2020",
  "Adobe",
  "Adobe's applications",
  "Intel-based Macs",
  "80% performance improvement"
]

Example output:
{
  "hyperedges": [
    {
      "text": "Apple launched the M1 chip in
2020.",
      "entities": ["Apple", "M1 chip", "2020"]
    },
    {
      "text": "Adobe optimized Adobe's
applications for the M1 chip after the chip was
launched.",
      "entities": ["Adobe", "Adobe's
applications", "M1 chip"]
    },
    {
      "text": "Adobe's applications on the M1
chip achieved up to an 80% performance
improvement compared with Adobe's applications
on Intel-based Macs.",
      "entities": [
        "Adobe",
        "Adobe's applications",
        "M1 chip",
        "80% performance improvement",
        "Intel-based Macs"
      ]
    }
  ]
}

Input:
Passage: {passage}
Entity list: {entities_json_list}
```

A.1.3 Question Decomposition

```
Task:
Break the input question into the smallest
number of retrieval-friendly sub-questions
needed to answer it reliably.

Main principle:
Decompose the question according to bridge facts,
 not according to surface length. Each sub-
question should normally resolve one unknown
bridge entity, one bridge relation, or the final
 requested attribute.

Guidelines:
1. If the question can be answered with a single
 retrieval step, return a list containing only
the original question.
2. For sequential reasoning, create one sub-
question for each bridge step when a later step
depends on the answer to an earlier step.
3. A good sub-question should usually do one of
the following:
   - identify a bridge entity;
   - identify a relation involving a known
bridge entity;
   - ask for the final attribute once the target
 entity is known.
4. Avoid sub-questions that require resolving
multiple new bridge entities at the same time.
5. Prefer two sub-questions for one-bridge
questions, three for two-bridge questions, and
four for three-bridge questions when needed.
6. Do not force every question into two sub-
questions. Use more steps when this makes
retrieval clearer.
7. Preserve the reasoning order. Later sub-
questions may refer to earlier answers using
phrases such as "that person", "that city", or "
that work".
8. For parallel or comparative questions,
retrieve the required attributes separately
before asking for the comparison.
9. Avoid redundant or overlapping sub-questions.
10. Return at most four sub-questions.

Output format:
Return only a valid JSON object:
{
  "sub_questions": [
    "first sub-question",
    "second sub-question"
  ]
}

Examples:
Question:
What county is Erik Hort's birthplace a part of?

Output:
{
  "sub_questions": [
    "What is Erik Hort's birthplace?",
    "Which county is that birthplace part of?"
  ]
}

Question:
When was the start of the battle of the
birthplace of the performer of III?

Output:
{
  "sub_questions": [
    "Who is the performer of III?",
    "What is that performer's birthplace?",
    "When did the battle at that birthplace
start?"
  ]
}

Input:
Question: {question}
```

A.1.4 Sub-query Evaluation

```
Task:
Given a sub-query, retrieved atomic-fact
hyperedges, and source passages, decide whether
the evidence answers the sub-query.

Output format:
Return only a valid JSON object:
{
  "status": "ANSWERED" or "UNANSWERED",
  "answer": "short sub-answer or null",
  "supporting_hyperedge_ids": [0, 1]
```

```
}

Guidelines:
1. Use ANSWERED only when the hyperedges and
passages support a direct answer.
2. The answer must be concise.
3. supporting_hyperedge_ids are 0-based indices
from the listed hyperedges.
4. Do not include explanations outside the JSON
object.

Sub-query: {sub_query}

Hyperedges:
{hyperedges_block}

Passages:
{passages_block}
```

A.1.5 Final Answer Integration

```
Task:
Given the original question and the sub-question
 answers produced during multi hop reasoning,
combine the intermediate answers into a single
final answer.

Guidelines:
1. The final answer must directly answer the
original question.
2. Use the available sub-question answers.
3. If one sub-answer is missing, still provide
the best short answer supported by the available
 information.
4. Output only the answer, without reasoning or
restating the question.
5. Return only a valid JSON object.

Output format:
{
  "final_answer": "..."
}

Original question:
{original_question}

Sub-questions and answers:
{qa_block}
```

A.1.6 QA Reader Fallback

```
System prompt:
As an advanced reading comprehension assistant,
your task is to analyze text passages and
corresponding questions meticulously. Your
response start after "Thought: ", where you will
 methodically break down the reasoning process,
illustrating how you arrive at conclusions.
Conclude with exactly one line formatted as "
FINAL_ANSWER: <answer>". The final answer must
be a concise answer span only, with no
explanation, no restated question, and no extra
text after it.

User prompt:
Wikipedia Title: {title_1}
{passage_text_1}

Wikipedia Title: {title_2}
{passage_text_2}

...

Retrieved propositions:
{numbered_propositions}

Retriever hypothesis: {retriever_hypothesis}

Question: {question}
Thought:
```

A.1.7 Answer Judge

```
Task:
Evaluate whether a model prediction correctly
answers the question by comparing it against the
 reference answer.

Decision rule:
Return [[yes]] if the prediction is factually
equivalent to the reference answer or contains
the reference answer. Return [[no]] otherwise.

Guidelines:
1. Accept paraphrases, aliases, abbreviations,
and equivalent date formats.
2. Accept a response that contains the required
answer even if it includes additional reasoning.
3. Return [[no]] if the response contains only
part of the required answer.
4. Return [[no]] if the response is broader,
narrower, or contradictory.
5. Return only [[yes]] or [[no]].

Question:
{question}

Reference answer:
{ground_truth}

Model prediction:
{prediction}

Is the model prediction correct?
```

A.2 Reader-Fallback Analysis

Reader fallback is triggered only when at least one decomposed sub-query remains UNANSWERED. Its frequency rises from $41/518 = 7.92\%$ on 2-hop questions to $44/166 = 26.51\%$ on 4-hop questions. The monotonic increase is consistent with a larger opportunity for an unresolved intermediate step; it does not by itself identify which component caused the failure. Across all depths, the fallback is used for $123/1{,}000 = 12.3\%$ of questions.

A.3 Hyperparameter Sensitivity

Relative to the default, accuracy changes by at most 1.3 points across the five tested settings, indicating that performance is not sharply sensitive within this local range. Changing α mainly shifts the balance between structural and query-conditioned transitions. Increasing α from 0.50 to 0.75 raises recall from 82.31 to 83.06 and F1 from 59.23 to 59.61, but slightly lowers accuracy from 71.60 to

Method	Category	Retrieval Unit	Multi-step	Main Role
Contriever	Dense Retrieval	Passage	No	Unsupervised dense retrieval baseline
BGE-M3	Dense Retrieval	Passage	No	Strong dense retriever and HyperProve seed retriever
HippoRAG2	Structured RAG	Passage + Triples	Yes	Associative graph retrieval baseline
PropRAG	Structured RAG	Propositions	Yes	Structured fact retrieval baseline
HyperGraphRAG	Structured RAG	Propositions	Yes	Hypergraph-structured retrieval baseline
HGRAG	Structured RAG	Propositions	Yes	Entity–passage hypergraph baseline
IRCoT	Iterative Retrieval	Passage	Yes	Retrieval interleaved with reasoning
T2RAG	Iterative Retrieval	Triple	Yes	Triplet-driven retrieval and reasoning
HopRAG	Iterative Retrieval	Passage	Yes	Retrieve–reason–prune baseline
ChainRAG	Iterative Retrieval	Passage + Sentence	Yes	Progressive retrieval baseline
HyperProve	Ours	Propositions	Yes	Answer-guided hypergraph expansion

Table 9: Summary of baseline methods and retrieval units. HyperProve differs from prior methods by combining atomic-fact hyperedges with answer-guided, stateful expansion over local hypergraph neighborhoods.

Depth	Fallbacks	Questions	Rate
2-hop	41	518	7.92%
3-hop	38	316	12.03%
4-hop	44	166	26.51%
Overall	123	1,000	12.30%

Table 6: Reader-fallback frequency by annotated reasoning depth on MuSiQue.

α	θ	Recall	EM	F1	Acc.
0.50	0.40	82.31	42.83	59.23	71.60
0.25	0.40	81.18	42.41	58.76	70.80
0.75	0.40	83.06	43.12	59.61	71.40
0.50	0.30	83.45	41.97	58.49	70.30
0.50	0.50	81.72	43.05	59.40	71.90

Table 7: MuSiQue sensitivity to structural-transition weight α and query-transition threshold θ. The first row is the default configuration.

71.40; decreasing it to 0.25 reduces both recall and accuracy to 81.18 and 70.80. The threshold θ shows a clearer recall–selectivity trade-off. Lowering it to 0.30 admits more query-relevant neighbors and gives the highest recall, 83.45, but reduces EM, F1, and accuracy. Raising it to 0.50 filters more aggressively, lowering recall to 81.72 while producing the highest accuracy, 71.90. Because no alternative improves retrieval and answer metrics consistently, we retain the balanced default $(\alpha, \theta) = (0.50, 0.40)$ rather than selecting a configuration post hoc.

A.4 Scalability Limits

The current implementation performs synonym linking as batched all-pairs comparisons and rebuilds global graph artifacts after corpus changes. We have not benchmarked web-scale corpora or dynamic index maintenance, so Table 5 should not be read as evidence for those settings. Approximate-nearest-neighbor synonym search and incremental updates are promising future implementation directions, but are not measured capabilities of the present system. Complementarily, recent embedding-model distillation methods such as EMO, SAMD, and TALAS could compress the encoder used for dense seeding and synonym matching (Truong et al., 2025; Tran et al., 2026; Dao et al., 2026b); this direction targets encoder-side retrieval cost rather than LLM-call cost and has not been evaluated with HyperProve.

Dataset	Questions	Passages	Entities	Hyperedges
2WikiMultiHopQA	1,000	6,119	57,906	55,032
HotpotQA	1,000	9,812	112,308	99,394
MuSiQue	1,000	11,656	117,894	105,542

Table 8: Dataset statistics for the 1,000-question evaluation subsets.

A.5 Dataset Statistics

We evaluate HyperProve on three open-domain multi-hop question answering benchmarks: MuSiQue (Trivedi et al., 2022), HotpotQA (Yang et al., 2018), and 2WikiMultiHopQA (Ho et al., 2020). Following PropRAG (Wang and Han, 2025), we use the corresponding testing set for each dataset to ensure a consistent and comparable evaluation protocol. For the reported experiments, each benchmark is evaluated on a 1,000-question subset from its testing set, and the corpus passages associated with that split are processed offline to construct HyperProve's atomic-fact hypergraph.

MuSiQue. MuSiQue is a multi-hop question answering benchmark designed to test compositional reasoning over multiple supporting passages. Questions in MuSiQue are constructed to reduce shortcut reasoning and require models to combine evidence from distinct reasoning steps before produc-

ing the final answer.

HotpotQA. HotpotQA is a widely used multi-hop question answering benchmark built from Wikipedia. It contains questions that require reasoning over multiple documents, including bridge-style questions, where an intermediate entity must be identified, and comparison questions, where information about multiple entities must be compared.

2WikiMultiHopQA. 2WikiMultiHopQA is an entity-centric multi-hop question answering benchmark constructed using Wikipedia and Wikidata. It emphasizes reasoning over structured entity relations and typically requires combining evidence across multiple linked facts or documents.

Table 8 summarizes the three datasets used in our experiments. These benchmarks cover complementary forms of multi-hop reasoning, including compositional reasoning, bridge reasoning, comparison reasoning, and entity-relation reasoning. In HyperProve, each extracted atomic fact is represented as one hyperedge connected to its participating entities and source passage.

A.6 Positioning Relative to Prior Work

Table 9 positions HyperProve relative to the evaluated baselines by comparing their retrieval units and multi-step control mechanisms. Dense retrievers rank passages in a single step, which makes them useful for seed retrieval but leaves them without an explicit cross-hop evidence state. Structured RAG systems add graph, fact-path, or hypergraph connectivity, but their expansion is primarily driven by static structure or pre-defined retrieval paths. Iterative systems introduce multi-step control, although their state is mainly textual or tied to passages, triples, or sentence-level evidence.

HyperProve combines these directions by retrieving over atomic-fact hyperedges, preserving n-ary entity context and source-passage provenance, and using intermediate answers to define the next local expansion frontier. This makes retrieval stateful, fact-centered, and structurally constrained while still allowing fresh dense seeds to recover evidence outside the carried frontier.

A.7 Case Study 1: Hidden-Bridge Reasoning

Figure 4 shows a hidden-bridge MuSiQue example where the question asks: "When was the start of the battle of the birthplace of the performer of III?" The original question does not explicitly mention the required bridge entities: *Stanton Moore*, *New Orleans*, or the *Battle of New Orleans*. This makes the example difficult for retrieval methods that mainly rank evidence according to surface similarity with the original question.

HyperProve decomposes the question into dependent sub-queries and carries intermediate answers forward as bridge entities. It first identifies *III* as Stanton Moore's third studio solo album, then uses *Stanton Moore* to retrieve evidence about his birthplace, *New Orleans*. The system then uses *New Orleans* as the next bridge entity to retrieve evidence about the *Battle of New Orleans* and its start date. This answer-guided evidence chain leads to the correct answer, *December 14, 1814*.

HippoRAG2, by contrast, over-emphasizes broad surface terms from the original question, such as *III*, *battle*, and *birthplace*. Because the key intermediate entities are not present in the original question, its associative retrieval process follows unrelated evidence and fails to recover the hidden bridge chain. It therefore produces the incorrect answer, *1 September 1054*. This case highlights the importance of answer-conditioned bridge propagation: in multi-hop QA, the next useful evidence is often not lexically similar to the original question, but becomes reachable only after an intermediate entity has been resolved.

A.8 Case Study 2: Cascading Answer State

The gold chain in this four-hop example is *California* → *Samuel Brannan* → *Sacramento* → *Rio Linda*, whereas HyperProve predicts *California* → *Francis J. Banfield* → *New York City* → *New Jersey*. The first hop resolves the location correctly, but the second accepts Francis J. Banfield as a plausible yet unsupported person. Because this answer conditions both the rewritten query and the structural frontier, the third hop follows evidence toward New York City rather than Sacramento. The final hop then retrieves locally coherent evidence around the wrong frontier and returns New Jersey instead of Rio Linda.

This case exposes the main risk of answer-guided retrieval: later evidence can remain coherent with a propagated state while becoming inconsistent with the original question. Confidence-aware propagation could defer uncertain bridges; provenance constraints could reject unsupported frontiers; and multi-frontier recovery could retain alternatives. These remain unevaluated future directions.

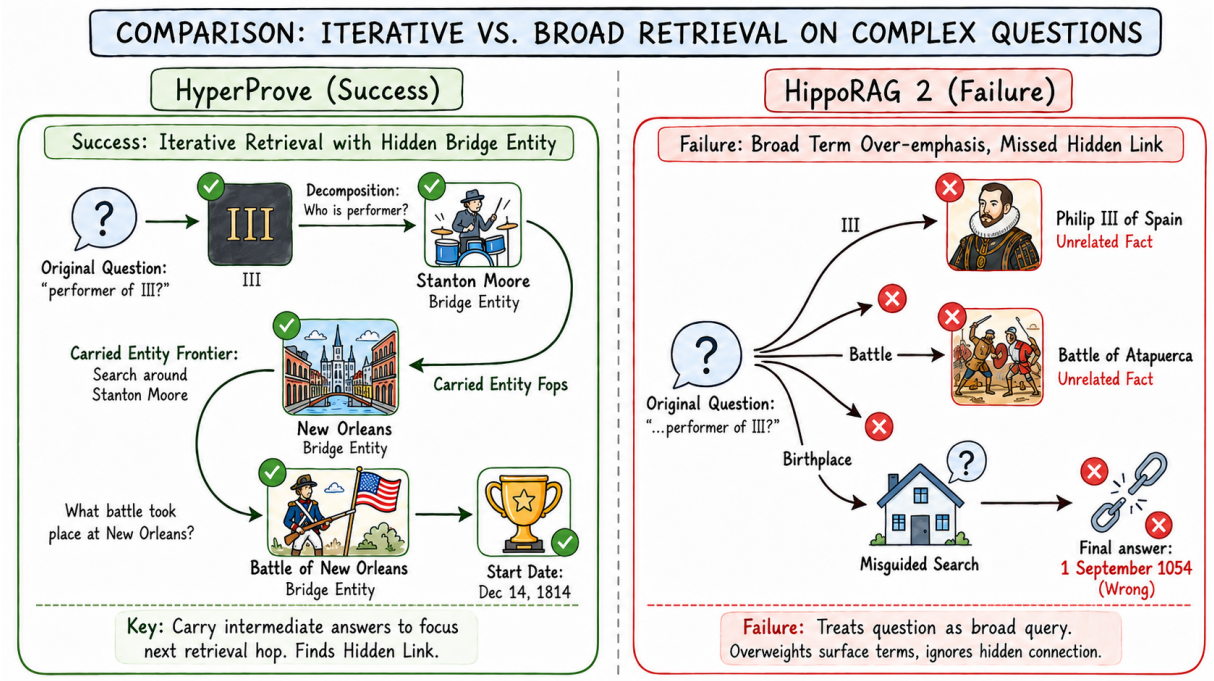


Figure 4: Qualitative comparison on a hidden-bridge MuSiQue example. HyperProve carries intermediate answers as bridge entities, enabling retrieval around *Stanton Moore*, *New Orleans*, and the *Battle of New Orleans*. In contrast, HippoRAG2 over-emphasizes broad surface terms from the original question and misses the hidden evidence chain.